Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project Report

Team leader: Nency Patel
Team members: Shreyansh Agarwal, Aarav Johri, Jal Savani
Code and data: `https://github.com/ndp5292/CMPSC488-Midterm-Project`

October 11, 2026

Abstract

We study whether a neural classifier can tell which large language model (LLM) family wrote a piece of text. From the public Chatbot Arena Human Preference 55K dataset we build a balanced set of 3,000 responses from three families (Claude, GPT and Gemini), split by prompt so that no prompt appears in more than one split. We train a TextCNN and a bidirectional LSTM and evaluate them on a held-out test set of 299 responses. From the response alone (RQ1), both models identify the family about 80% of the time against a chance level of 33%: the CNN reaches 0.803 ± 0.008 accuracy over three seeds and the LSTM 0.799 in a single run (95% bootstrap interval 0.753 to 0.843). The two models were trained under different settings, so we report them as performing similarly rather than as a controlled comparison. Claude is recognized almost perfectly, while most errors are GPT and Gemini confused with each other. The user's prompt alone carries no usable signal (RQ2): both models stay at chance, and adding the prompt to the response does not change accuracy. For RQ4 we analyze what separates the families. Formatting is the strongest single cue (Gemini uses bold text in 39% of responses, Claude in 1%), followed by recurring phrases such as Claude's "Here is" openings and "let me know if" closings. Formatting tokens are 11% of the LSTM's input but receive 43% of its attention, and removing formatting at test time lowers the LSTM's accuracy by 24 points. A CNN retrained without formatting loses only 5 points. Because these two experiments use different models and protocols, we read them as complementary rather than directly comparable. A simple TF-IDF logistic regression reaches similar accuracy, so the two neural models do not beat a bag of word bigrams at this data size.

1 Problem definition

LLMs now write a large share of the text people read online, which raises a practical question: given a piece of text, can we tell which model produced it? Different LLMs are trained on different data, with different instruction-tuning and preference-tuning procedures, so they may leave consistent stylistic and structural "fingerprints" in their answers.

We frame this as supervised multi-class classification. Each example is a triple $(y, x_{\text{in}}, x_{\text{out}})$, where $y \in \{\text{Claude}, \text{GPT}, \text{Gemini}\}$ is the LLM family, x_{in} is the user's prompt and x_{out} is the model's response. A classifier f maps text to a distribution over the three families. We investigate:

- **RQ1:** Can we identify which LLM generated a response? Here f sees only x_{out}.
- **RQ2:** Does the user's prompt help? We compare $f(x_{\text{in}})$, $f(x_{\text{out}})$ and $f(x_{\text{in}}, x_{\text{out}})$.
- **RQ4 (extra credit):** What characteristics distinguish the families, and which signals do the classifiers use?

We did not investigate RQ3 (generalization across tasks) because the dataset has no task or domain labels.

Metrics. We report accuracy and macro-F1, the unweighted mean of the per-class F1 scores. Macro-F1 treats the three families equally even though the test set is slightly unbalanced (114

Claude, 88 GPT, 97 Gemini). Chance accuracy for a uniform random guess is 1/3. Always predicting the most common training class (GPT) gives 0.294 test accuracy, and always predicting the most common test class (Claude) would give 0.381.

2 Dataset curation

2.1 Source

We use the Chatbot Arena Human Preference 55K dataset (`https://huggingface.co/datasets/lmarena-ai/arena-human-preference-55k`), released by LMArena under the Apache 2.0 license [1]. Each row is an anonymous "battle": a user writes a prompt, two randomly chosen models answer it, and the user votes for the better answer. Each row records the prompt, both responses and both model names (`model_a`, `model_b`). We use only the text and the model names, not the votes. The data is publicly released for research use, and we did not collect any new user data or call any LLM API.

This source has a useful property for RQ2: in Arena, the user writes the prompt before knowing which models will answer, and models are assigned to battles at random. The prompt should therefore carry little information about which model answered it.

2.2 Construction

The preprocessing notebook (`preprocessing/Midterm_Dataset_Preprocessing.ipynb`) performs the following steps.

1. **Family labels.** A model name containing `gpt` is mapped to GPT, `claude` to Claude and `gemini` to Gemini (case-insensitive). Other models are dropped. Each family therefore mixes several model versions (for example different GPT-3.5 and GPT-4 releases), so we classify families, not individual models.
2. **Text conversion.** Prompts and responses are stored as JSON lists of turns; we join the turns of a conversation with newlines. Each battle contributes up to two examples, one per side.
3. **Cleaning.** We remove rows with a missing or empty prompt or response and exact duplicates of (family, prompt, response).
4. **Balancing.** We sample 1,000 responses per family with a fixed random seed (42), giving 3,000 examples. The full cleaned pool is larger, but a balanced set of this size keeps training cheap and makes chance level exactly 1/3.
5. **Splitting.** We split 80/10/10 with `GroupShuffleSplit`, grouping on the normalized prompt (lowercased, stripped of surrounding whitespace). The notebook asserts that the three splits share no prompt.

Grouping by prompt matters. Many Arena prompts are repeated ("hello", common test questions), and a random split would place the same prompt, and often a near-identical answer, in both training and test sets. That would inflate test accuracy, especially for the input-only setting of RQ2.

Because each Arena battle has two answers, some prompts were answered by two families. This holds for 67 prompts (140 responses): 56 in training, 7 in validation and 4 in test. For these prompts, the prompt alone cannot determine the label even in principle.

Split	Claude	GPT	Gemini	Total	Unique prompts
Train	785	808	807	2,400	2,320
Validation	101	104	96	301	290
Test	114	88	97	299	291
Total	1,000	1,000	1,000	3,000	2,901

Table 1: Examples per split and family. Grouping by prompt keeps every prompt inside a single split, which is why the per-family counts are only approximately balanced within each split.

2.3 Characteristics of the three families

Table 2 and Figure 1 summarize the responses. Lengths overlap heavily: the median Claude, GPT and Gemini responses have 176, 164 and 208 words, and all three distributions span from a few words to over a thousand. The prompts are also similar across families (median 16 to 18 words), as expected from random model assignment. Formatting and phrasing differ much more. Gemini uses bold text in 39% of its responses against 11% for GPT and 1% for Claude. Claude uses bullet lists most often (44%) and starts 38% of its responses with "Here is" or "Here are", a phrase GPT and Gemini almost never open with. GPT has the fewest bullet lists and newlines but uses numbered lists as often as Gemini. Section 5 analyzes these differences in detail.

	Claude	GPT	Gemini
Response length, median words (IQR)	176 (60 to 246)	164 (58 to 332)	208 (40 to 328)
Prompt length, median words	17	18	16
Newlines per 100 words, median	6.3	3.9	6.4
Lexical diversity, MATTR (100 to 400 words)	0.774	0.753	0.713
Contains bold **	1.3%	10.9%	39.3%
Contains a bullet list	43.8%	10.4%	34.5%
Contains a numbered list	15.3%	29.8%	30.2%
Contains a markdown header	2.3%	5.7%	4.3%
Contains a code block	9.1%	11.0%	12.0%
Starts with "Here is/are"	37.6%	0.4%	0.1%
Starts with "Sure/Certainly/Of course"	0.8%	8.1%	2.5%
Contains "let me know"	19.6%	0.9%	1.5%
Contains "remember"	0.5%	11.2%	2.7%
Contains an apology ("I apologize", "I'm sorry")	8.9%	4.3%	1.6%

Table 2: Response characteristics per family over all 3,000 examples. MATTR is the moving-average type-token ratio with a 50-word window [2], computed only on responses of 100 to 400 words so that length does not drive the comparison (588, 470 and 505 responses).

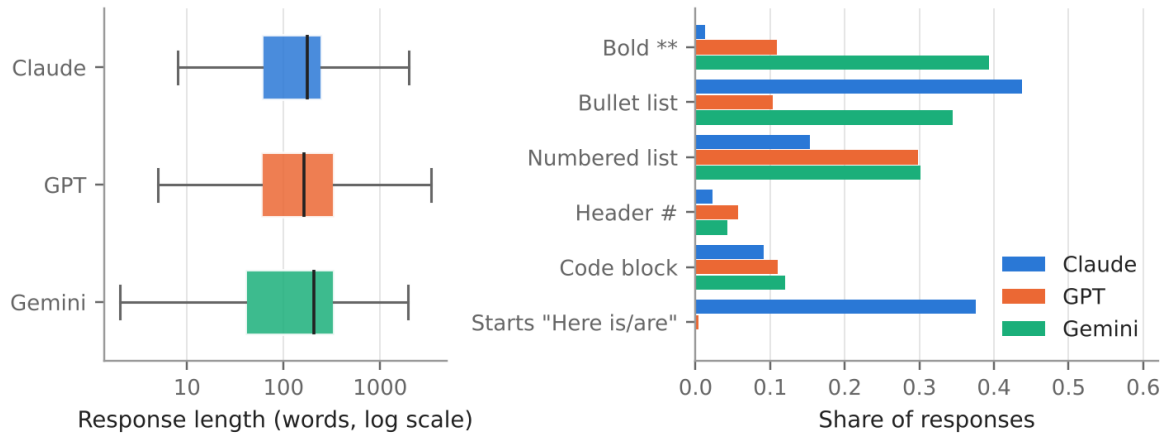


Figure 1: Left: response length per family (boxes show the quartiles, whiskers the 1.5 IQR range, log scale). Right: share of responses with each formatting feature or opening phrase.

3 Models and training

Both models are trained from scratch on the same fixed splits. The CNN is in `cnn/cnn_classifier.py` and the LSTM in `rnn/lstm_classifier.py`. Table 3 lists all hyperparameters side by side. The two models were developed separately and differ in many settings beyond the architecture, including class-weighted loss, tokenizer, case handling, sequence length, optimizer and number of seeds; Section 3.4 and Section 5.6 discuss what this means for comparing them.

3.1 Tokenization and vocabulary

Formatting is part of a model's style, so neither tokenizer strips it. The CNN uses the regular expression `\n|\w+|[^\w\s]`, which makes words, single punctuation marks and newlines separate tokens, and keeps the original case. The LSTM lowercases the text and keeps markdown markers as whole tokens: bold `**`, header marks (`#` to `######`), code fences, newlines (`<nl>`) and numbered-list markers such as "1." (`<num_item>`). Words keep contractions ("don't"), and other numbers become `<num>`. For both models the vocabulary is built from the training split only (at most 30,000 tokens, minimum frequency 2), and unseen tokens map to `<unk>`. The CNN pads or truncates responses to 256 tokens and prompts to 64. The LSTM truncates responses to 512 tokens and prompts to 128, and uses packed sequences so that padding never enters the recurrence.

3.2 CNN

We use the TextCNN of Kim [5]. Each token is embedded in 128 dimensions. Four parallel 1D convolutions with window widths 2, 3, 4 and 5 (100 filters each) slide over the sequence, followed by ReLU and max-over-time pooling, which keeps the strongest response of each filter anywhere in the text. The four pooled vectors are concatenated (400 dimensions), passed through dropout 0.5 and a linear layer with three outputs. For the input+output setting the CNN uses two encoders, one for the prompt and one for the response, and concatenates their features before the linear layer. The output-only model has 3.04 million parameters, most of them in the embedding table.

3.3 LSTM

The LSTM [3] embeds tokens in 128 dimensions and reads them with a one-layer bidirectional LSTM with 128 hidden units per direction, so each position t has a 256-dimensional state $h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t]$ that depends on the whole sequence. The states are summarized in two complementary ways:

$$\alpha_t = \frac{\exp(w^\top h_t)}{\sum_{s=1}^{T} \exp(w^\top h_s)}, \qquad a = \sum_{t=1}^{T} \alpha_t h_t, \qquad m = \max_t h_t \text{ (element-wise)},$$

where attention pooling a learns which positions matter and max pooling m picks up strong local cues wherever they occur. The vector $[a; m]$ (512 dimensions) goes through dropout 0.4, a 256-unit ReLU layer, dropout 0.4 and a linear layer with three outputs. For the input+output setting the prompt and response are joined into one sequence, prompt `<sep>` response, and read by the same LSTM. The output-only model has 2.48 million parameters. We use the attention weights α_t in Section 5 to see which tokens the model focuses on.

	CNN	LSTM
Case	kept	lowercased
Max length (response / prompt)	256 / 64 tokens	512 / 128 tokens
Embedding	128, trained from scratch	128, trained from scratch
Encoder	conv widths 2, 3, 4, 5 × 100 filters	BiLSTM, 1 layer, 128 per direction
Pooling	max over time	attention + max over time
Classifier head	dropout 0.5, linear	dropout 0.4, 256 ReLU, dropout 0.4, linear
Input + output	two encoders, concatenated	one sequence with `<sep>`
Parameters (output only)	3.04M	2.48M
Optimizer	Adam [6], lr 10^{-3}	AdamW [7], lr 2×10^{-3}
Weight decay	10^{-4}	10^{-4}
Batch size	64	32
Loss	class-weighted cross-entropy	cross-entropy
Tokenizer	words, punctuation, newlines	markdown-aware (see Section 3.1)
Gradient clipping	none	norm 1.0
LR schedule	constant	halve when val macro-F1 stalls 1 epoch
Max epochs / early stopping	20 / patience 4 on val macro-F1	15 / patience 4 on val macro-F1
Seeds	0, 1, 2 (mean ± sd reported)	42 (single run)

Table 3: Model and training settings. Both models were implemented in PyTorch [10] and trained on CPU.

3.4 Training and model selection

Both models minimize cross-entropy (class-weighted for the CNN, unweighted for the LSTM) and are evaluated on the validation set after every epoch. We keep the epoch with the highest validation macro-F1 and stop when it has not improved for four epochs. The selected model is evaluated on the test set exactly once; no hyperparameter was tuned on the test set. The CNN was trained with three seeds per setting, and we report the mean and standard deviation. The LSTM is much slower to train on CPU: one output-only LSTM run took about 40 minutes (about 160 seconds per epoch), while all 18 CNN runs together took about an hour. Each LSTM setting was therefore run once, with seed 42. To give the LSTM results an uncertainty estimate anyway, we compute 95% bootstrap intervals over the test examples (10,000 resamples).

Comparison caveat. Throughout this report, CNN numbers are the mean of three seeds and LSTM numbers are a single run (seed 42). The two models also differ in the settings listed in Table 3, notably the CNN's class-weighted loss and simpler tokenizer, case handling, sequence lengths,

epochs and optimizer. Comparisons between them therefore describe the two systems as configured, not convolution versus recurrence alone.

Additional baselines. To put the neural results in context we also train three simple scikit-learn models [11] on the same splits: logistic regression on TF-IDF word unigrams and bigrams, logistic regression on TF-IDF character 2- to 5-grams, and logistic regression on 21 hand-crafted style features (Section 5.2). The code for these and all other analyses in this report is in `analysis/report_analysis.py`.

4 Results

4.1 RQ1: identifying the LLM from its response

Model (output only)	Accuracy	Macro-F1
Chance (uniform guess)	0.333	0.333
Majority class (always GPT)	0.294	0.152
CNN (mean of 3 seeds)	0.803 ± 0.008	0.793 ± 0.009
LSTM (single run, seed 42)	0.799 [0.753, 0.843]	0.790
Logistic regression, 21 style features	0.719	0.714
Logistic regression, TF-IDF character 2- to 5-grams	0.793	0.787
Logistic regression, TF-IDF word 1- and 2-grams	0.816	0.811

Table 4: RQ1 test results (299 responses). The interval for the LSTM is a 95% bootstrap interval over test examples.

Both required models identify the family from the response alone about 80% of the time, far above the 33% chance level (Table 4). The two neural models performed similarly, though under different settings (Table 3). The difference is well within sampling error: the CNN's mean accuracy of 0.803 lies well inside the LSTM's bootstrap interval, one standard error of accuracy with 299 test examples is about 2.3 points. We therefore make no claim that either model is better. A TF-IDF logistic regression reaches the same level (0.816), which we return to in Section 5.6.

	CNN (mean of 3 seeds)			LSTM (single run, seed 42)		
Family	Precision	Recall	F1	Precision	Recall	F1
Claude	0.929	0.918	0.924	0.972	0.921	0.946
GPT	0.700	0.731	0.714	0.670	0.739	0.703
Gemini	0.753	0.732	0.742	0.734	0.711	0.723

Table 5: Per-class test metrics for the output-only models.

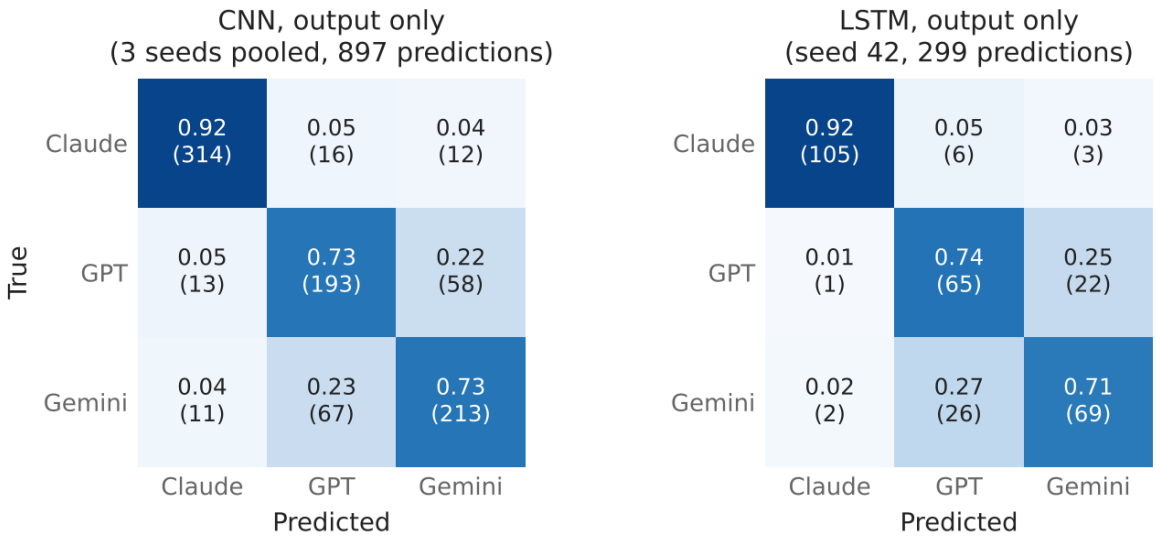


Figure 2: Test confusion matrices for the output-only models, normalized by row (true class); counts in parentheses. The CNN matrix pools the three seeds.

The per-class results (Table 5, Figure 2) show the same structure for both models. Claude is the easiest family, with F1 above 0.92 and 92% recall for both models. Nearly all remaining errors are between GPT and Gemini: for the LSTM, 48 of the 60 test errors are GPT predicted as Gemini (22) or Gemini predicted as GPT (26); for the CNN, 125 of the 177 pooled errors are of this kind. Claude is almost never predicted for a GPT or Gemini response. Section 5 explains this pattern: Claude's style differs from the other two in several consistent ways, while GPT and Gemini differ mainly in the use of bold text.

Training behavior. Figure 3 shows the output-only LSTM's training curves. Training macro-F1 climbs to 0.98, but validation loss reaches its minimum (0.62) at epoch 3 and then rises to 1.18, a sign of overfitting and of increasingly over-confident predictions. Validation macro-F1 nevertheless keeps improving slowly, from 0.73 at epoch 3 to 0.76 at epoch 15, so model selection by macro-F1 keeps the last epoch. The model gets more examples right while being too confident on the ones it gets wrong. The CNN behaved similarly: its runs stopped after 12 to 17 epochs, with best validation macro-F1 of 0.766 to 0.803 across seeds.

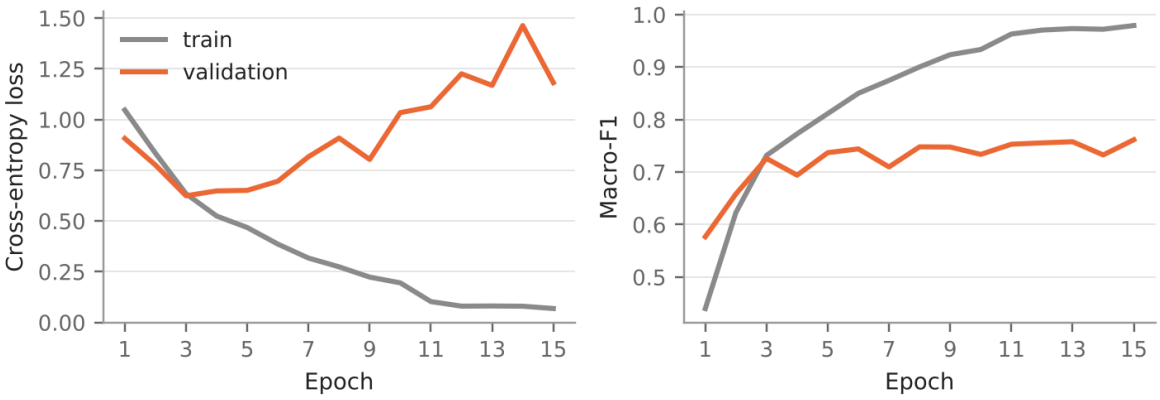


Figure 3: Output-only LSTM: training and validation loss (left) and macro-F1 (right) per epoch.

4.2 RQ2: does the user's prompt help?

Setting	CNN (mean of 3 seeds) Accuracy	CNN (mean of 3 seeds) Macro-F1	LSTM (single run, seed 42) Accuracy	LSTM (single run, seed 42) Macro-F1
Input only	0.313 ± 0.014	0.301 ± 0.022	0.348	0.345
Output only	0.803 ± 0.008	0.793 ± 0.009	0.799	0.790
Input + output	0.804 ± 0.004	0.796 ± 0.004	0.793	0.784

Table 6: RQ2 test results. Chance accuracy is 0.333. A TF-IDF logistic regression on the prompt alone gives 0.334 accuracy.

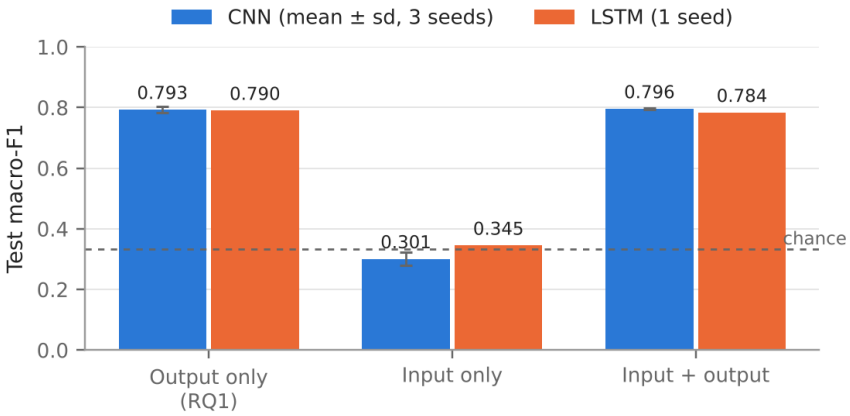


Figure 4: Test macro-F1 by input setting. CNN bars show the mean over three seeds with ±1 standard deviation; LSTM bars are single runs (seed 42).

(i) Can the LLM be predicted from the prompt alone? No. Both models stay at chance level with the prompt alone (CNN 0.313, LSTM 0.348 accuracy), and so does a TF-IDF logistic regression on the prompt (0.334). The best input-only LSTM epoch was the first, and its validation macro-F1 (0.42) did not carry over to the test set, so the model found no signal that generalizes.

(ii) Does including the prompt improve classification? No. Adding the prompt changes accuracy by +0.1 points for the CNN and −0.7 points for the LSTM, both far smaller than the noise level. A paired comparison of the two LSTM models on the same test examples confirms this: the output-only model is right and the input+output model wrong on 20 examples, and the reverse on 18 (exact McNemar test [8], $p = 0.87$; 95% bootstrap interval for the accuracy difference −3.3 to +4.7 points). The two models agree on 86% of test predictions, so the extra input mostly leaves the decision unchanged.

(iii) When would input-only classification work, and why not here? A prompt can only predict the model if prompts and models are statistically dependent. That happens when data is collected per model: for example when each model is queried with a different prompt set, when users of a given product ask different kinds of questions, or when prompts contain traces such as a system prompt, a model name or a language. In those cases an input-only classifier would learn the collection process rather than any model fingerprint, and its accuracy would be a warning sign of leakage. Our data avoids this by construction. Arena users write prompts before models are assigned at random, prompt lengths are nearly identical across families (Table 2), 67 prompts were answered by two different families, and splits are grouped by prompt so that memorized prompts

cannot help on the test set. The chance-level input-only results are therefore what we should expect, and they serve as a check that the splits do not leak.

5 In-depth analysis: what distinguishes the LLMs? (RQ4)

Accuracy shows that the families can be distinguished; this section asks why. We combine four views: descriptive statistics, the most distinctive tokens per family, classifiers restricted to specific feature groups, and experiments that remove signals from the text. The retraining ablations in Section 5.3 were run on the CNN only; the LSTM was analyzed through attention weights and test-time signal removal.

5.1 Distinctive tokens

To find the tokens most characteristic of each family we compare each family with the other two on the training set, using the log-odds ratio with an informative Dirichlet prior [9]. The method gives a z-score per token and avoids over-weighting rare words. We use the LSTM tokenizer so that formatting tokens are included.

Family	Most distinctive tokens (z-score)
Claude	numbers `<num>` (21.6), so (19.0), here (17.6), like (16.7), know (15.0), some (14.9), bullet `-` (14.5), let (14.3), etc (14.2), me (13.5), summary (12.1), ! (12.0)
GPT	`<` (14.0), inline code ‘ (13.0), it's (12.5), comma (11.8), might (11.7), ; (10.8), numbered item `<num_item>` (10.3), remember (9.9), here's (9.2), the (9.2), parentheses (9.2)
Gemini	bold `**` (61.4), `*` (32.7), : (27.9), $ (17.1), newline `<nl>` (12.1), used (10.3), \ (10.3), of (8.3), variety (8.3), is (8.0)

Table 7: Most distinctive tokens per family on the training split (log-odds ratio with informative Dirichlet prior, family vs. the other two).

The lists separate cleanly into structural and lexical cues (Table 7). Gemini's fingerprint is dominated by markdown: `**` has by far the largest z-score in the whole analysis, followed by single asterisks (bullets), colons (as in “Term: explanation”) and the $ and \ of LaTeX-style math. Claude's tokens are conversational: *here*, *so*, *like*, *some*, *etc*, and the words *let*, *me* and *know*, which come from closings such as “Let me know if you have any other questions.” This phrase appears in 19.6% of Claude responses and in under 2% of GPT or Gemini responses. GPT's tokens point to a more formal register (semicolons, parentheses, *might*), more inline code, numbered lists, and the word *remember*, which appears in 11.2% of GPT responses (“Remember to…”) against 0.5% for Claude.

5.2 Structural versus linguistic signals

To test whether the signal is structural, linguistic or both, we train logistic regressions on 21 handcrafted features in two groups. Structural features (9) are length in words, characters and lines, newlines per 100 words, and the presence of bold, bullet lists, numbered lists, headers and code blocks. Linguistic features (12) are mean word and sentence length, lexical diversity (MATTR), opening phrases (“Here is/are”, “Sure”, apologies), apologies anywhere, “as an AI”, and the rates of question marks, exclamation marks, colons and commas.

Features (logistic regression)	Number of features	Accuracy	Macro-F1
Structural only	9	0.612	0.614
Linguistic only	12	0.592	0.594
Structural + linguistic	21	0.719	0.714
TF-IDF word 1- and 2-grams	63,173	0.816	0.811

Table 8: Test results of classifiers restricted to feature groups.

Each group alone gets about 60% accuracy, far above chance, and together they reach 72% (Table 8). The groups are complementary rather than redundant: the combined model is ten points better than either. The largest coefficients of the combined model match the descriptive statistics: "starts with Here is/are" and bullet lists for Claude, bold text and newline density for Gemini, and response length, "Sure" openings, numbered lists and headers for GPT. The remaining gap to TF-IDF and the neural models (about 8 points) comes from word choice that 21 features cannot capture. So the answer to RQ4 (ii) is both: formatting is the strongest single cue, but the full fingerprint also needs phrasing and vocabulary.

Lexical diversity also differs. Among responses of comparable length (100 to 400 words), Gemini has clearly lower MATTR (0.713) than Claude (0.774) and GPT (0.753), consistent with its more templated, list-heavy answers that repeat terms such as "used" and "variety of".

5.3 Removing signals

We remove signals in two different ways that answer two different questions (Figure 5). The two experiments use different models, so they are not a controlled comparison of retraining against test-time removal.

- **Retraining without the signal (CNN only).** The CNN was retrained and tested on text with (a) newlines and markdown symbols removed, (b) all punctuation removed, or (c) only the first 50 tokens kept. This asks: can the families still be told apart without this signal?
- **Removing the signal only at test time (LSTM only).** We take the trained output-only LSTM (single run, seed 42) and remove the same signals only from the test responses, using the same rules. This asks: how much does the trained model rely on this signal?

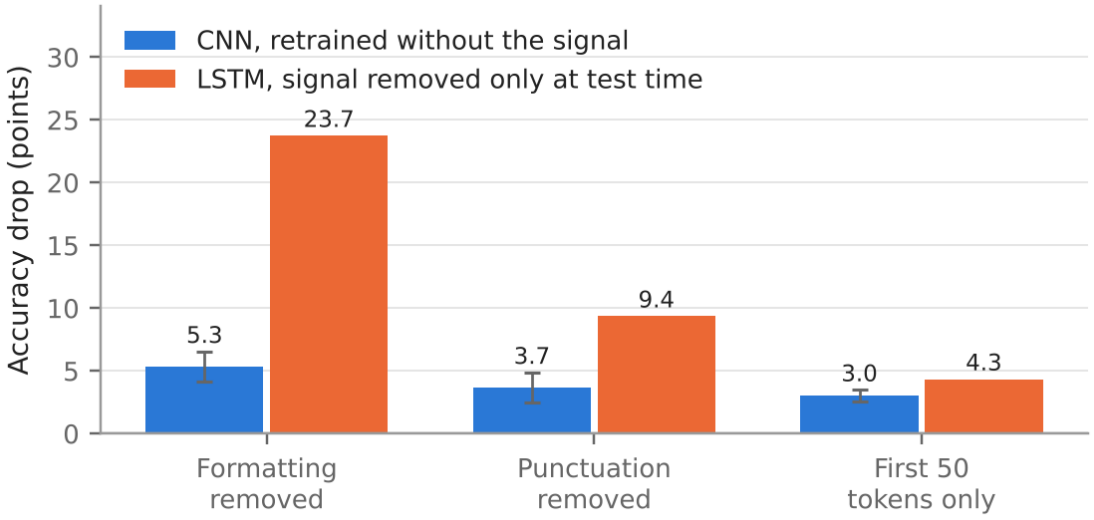


Figure 5: Accuracy drop relative to the full-text model when a signal is removed. CNN bars show retrained models (mean ± sd over three seeds); LSTM bars show the single trained model (seed 42) applied to modified test text.

Signal removed	CNN retrained, accuracy (mean ± sd, 3 seeds)	LSTM at test time, accuracy (single run, seed 42)
None (full text)	0.803 ± 0.008	0.799
Formatting (newlines and markdown)	0.749 ± 0.012	0.562
All punctuation	0.766 ± 0.012	0.706
Only the first 50 tokens kept	0.773 ± 0.005	0.756

Table 9: Test accuracy after removing signals.

The two experiments answer different questions (Table 9). When the CNN is retrained without formatting it loses only 5 points, so other cues (wording, phrasing, punctuation) can partly stand in for it. The trained LSTM, however, relies on formatting. With formatting removed at test time its accuracy falls by 24 points to 0.562, and its predictions collapse toward GPT, the least formatted family (193 of 299 predictions). Gemini recall drops from 0.71 to 0.26 and Claude recall from 0.92 to 0.58. The trained LSTM depends on formatting at test time, while the retrained CNN shows that formatting is not strictly necessary for about 75% accuracy. Because the two experiments use different models, we cannot say how much of the gap comes from the architecture and how much from the protocol (retraining versus test-time removal). Removing punctuation hurts less (4 points retrained, 9 at test time).

Keeping only the first 50 tokens costs just 3 to 4 points, so much of the fingerprint appears at the start of a response, where Claude's "Here is" openings and Gemini's bold headings sit. Accuracy still grows with length (Figure 6): the LSTM is right on 72% of the shortest quarter of test responses (at most 55 words) and on 83% to 87% of responses longer than 167 words, since a longer text gives more chances to show a characteristic habit.

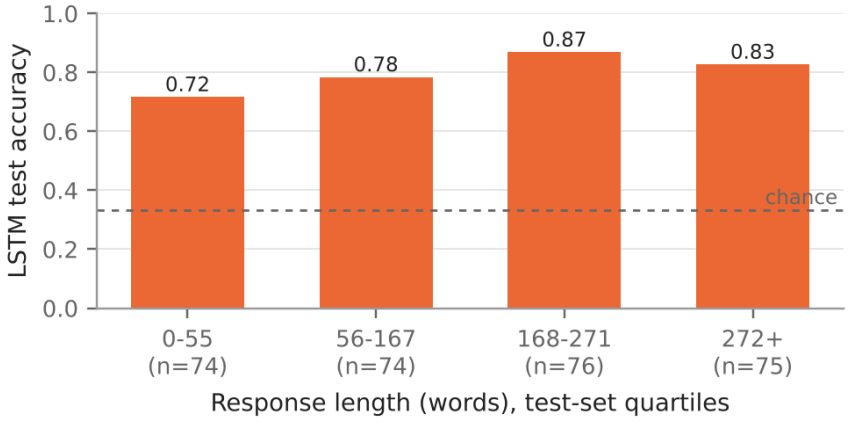


Figure 6: Output-only LSTM test accuracy by response length (quartiles of the test set).

5.4 What the LSTM attends to

The attention weights α_t show where the LSTM's attention pooling puts its weight. Formatting tokens (newlines, **, *, -, #, code fences, list markers and colons) make up 11.0% of the test tokens

but receive 43.2% of the total attention. The first 10% of each response receives 22.9% of the attention, about twice its share. Ranking tokens by their average attention relative to a uniform weighting gives a picture that matches the token analysis. For responses predicted as Gemini the most-attended token is ** (16 times the uniform weight). For GPT they include **, *remember*, *sure*, *however*, *please* and *note*. For Claude the top tokens are *any* (18 times uniform), *have*, *need* and *apologize*, the building blocks of Claude's closing "Let me know if you have any other questions" and its apologies. Attention weights are not a complete explanation of a model's decision [4], but here they agree with three independent analyses (the descriptive statistics, the distinctive tokens and the test-time removal experiment).

5.5 Why GPT and Gemini are confused

Bold text is the main thing that separates Gemini from GPT, and not every Gemini response uses it. Among the 37 Gemini test responses that contain bold text, the LSTM recognizes 34 (recall 0.92). Among the 60 that do not, it recognizes only 35 (recall 0.58) and labels 23 of them as GPT. Conversely, only 2 of the 22 GPT responses misclassified as Gemini contain bold text, and only 3 of the 26 Gemini responses misclassified as GPT do. Without bold text, GPT and Gemini look alike on our other measures: similar lengths, numbered lists in about 30% of responses each, and similar rates of code blocks and headers (Table 2). Claude, by contrast, has several independent markers ("Here is" openings, "let me know" closings, bullet lists, apologies, almost no bold text), so it stays recognizable even when one marker is missing.

5.6 Comparing the CNN, the LSTM and the baselines

The CNN (mean of three seeds), the LSTM (single run) and a TF-IDF logistic regression all reach about 80% accuracy (0.803, 0.799 and 0.816), within one standard error of each other. The LSTM reads the whole response in order and the CNN only sees windows of two to five tokens, yet neither beats a bag of word bigrams. This is consistent with much of the signal being available from words, short phrases and formatting markers, but we did not test this directly. The LSTM is somewhat better on Claude (F1 0.946 against 0.924) and slightly worse on GPT and Gemini, but each of these differences is only a few test examples.

The two models also differ in more than architecture (Table 3). The CNN keeps case, uses a class-weighted loss and a simpler tokenizer (words, punctuation and newlines only), and uses 256-token responses, 20 epochs and Adam. The LSTM lowercases, uses an unweighted loss and a markdown-aware tokenizer, and uses 512-token responses, 15 epochs and AdamW. The comparison is therefore between the two systems as built, not a controlled test of convolution against recurrence, and the two models performed similarly under different settings. With 2,400 training examples, 2.5 to 3 million parameters is a lot, and the overfitting in Figure 3 suggests that the neural models are limited by data rather than capacity.

6 Lessons learned

How the data is split decides what the results mean. The most important data decision was guarding against leakage. Arena contains many repeated prompts, and a random split would have placed the same prompt in training and test data. Grouping the split by prompt, and asserting that no prompt is shared, is what makes the input-only results meaningful. The chance-level input-only accuracy is itself a useful check: if it had been high, we would have looked for a leak before believing it.

Always compare against a simple baseline. It is natural to expect a sequence model to beat a CNN, and both to beat a linear model. In fact a TF-IDF logistic regression, trained in seconds, matches both neural models. Without that baseline it would be easy to over-interpret small differences between the CNN and the LSTM, or credit the LSTM's sequence modeling for a signal that a bag of words already captures.

Preprocessing choices are modeling choices. A standard text-cleaning pipeline (lowercasing, stripping punctuation and markdown) would have removed the strongest signal in the data. Keeping newlines and markdown markers as tokens was one of the most important decisions in the project, and the signal-removal experiments show how much it matters.

Different experiments answer different questions. Removing formatting from a retrained CNN costs 5 points, while removing it from a trained LSTM at test time costs 24. These results look contradictory but need not be: the first measures whether the information is available elsewhere in the text, the second whether a trained model depends on it. Because they also use different models, a cleaner design would apply both protocols to both models. Being precise about which question an experiment answers is necessary to interpret them, and it has practical consequences: a detector trained on formatted text can be fooled by simply removing the formatting.

Loss and accuracy can disagree. Validation loss started rising after three epochs while validation macro-F1 kept improving. Selecting on loss would have stopped training early with a worse classifier, while selecting on macro-F1 kept a more accurate but over-confident model. Watching both curves is necessary to understand what training is doing.

Small test sets need humility. With 299 test examples, one standard error of accuracy is about 2.3 points, larger than most of the differences between settings and models. Several seeds (CNN), bootstrap intervals and a paired McNemar test (LSTM) are what keep a report from claiming differences that are not there. In hindsight, we would also run the LSTM with three seeds and the same input settings as the CNN; we ran it once because each run took about 40 minutes on CPU.

Working as a team. Fixing the splits once, committing them to the repository and having every model read the same files made the results evaluable on identical test examples, although not under identical training settings, even though the models were developed separately. Keeping each model in its own folder with its own README, requirements and results also made it easy to combine the work at the end. What we would do differently is agree on shared settings (seeds, length limits, loss weighting, tokenizer, result format) before training, rather than discovering differences when comparing results.

Limitations and next steps. Each family mixes several model versions, so we learn a family-level fingerprint that may change with new releases. The classes are balanced by construction, which is not the natural distribution, and the Arena prompts are mostly short, general questions. The LSTM results come from one seed and the CNN and LSTM use different settings. Natural next steps are RQ3 with real task labels, matched multi-seed comparisons of the two architectures, applying both signal-removal protocols to both models, pretrained embeddings or transformer encoders, version-level labels, and robustness tests against paraphrasing and formatting changes, which our results suggest would be the easiest way to evade a detector.

7 Conclusion

A CNN and a bidirectional LSTM trained on 2,400 Arena responses identify whether a response came from Claude, GPT or Gemini about 80% of the time, against 33% chance. The two models performed similarly, although they were trained under different settings. The prompt alone carries no signal in this data, and adding it to the response does not help. The fingerprints are a mix of

structure and wording: Gemini's heavy use of bold markdown, Claude's "Here is" openings, "let me know" closings and bullet lists, and GPT's plainer, more formal style with "remember" and numbered lists. Claude is recognized almost perfectly, while GPT and Gemini are mostly confused when Gemini does not use bold text. The trained LSTM relies heavily on formatting, but a CNN retrained without it still reaches 75% accuracy, so formatting is not the only usable cue.

Use of LLM tools. AI assistants were used for coding help, debugging, analysis and drafting. All code, results and text were checked and are understood by the team.

References

[1] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez and I. Stoica. Chatbot Arena: An open platform for evaluating LLMs by human preference. In *Proceedings of ICML*, 2024.

[2] M. A. Covington and J. D. McFall. Cutting the Gordian knot: The moving-average type-token ratio (MATTR). *Journal of Quantitative Linguistics*, 17(2):94–100, 2010.

[3] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[4] S. Jain and B. C. Wallace. Attention is not explanation. In *Proceedings of NAACL-HLT*, 2019.

[5] Y. Kim. Convolutional neural networks for sentence classification. In *Proceedings of EMNLP*, 2014.

[6] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proceedings of ICLR*, 2015.

[7] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *Proceedings of ICLR*, 2019.

[8] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[9] B. L. Monroe, M. P. Colaresi and K. M. Quinn. Fightin' words: Lexical feature selection and evaluation for identifying the content of political conflict. *Political Analysis*, 16(4):372–403, 2008.

[10] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in NeurIPS*, 2019.

[11] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.